SUNWAY UNIVERSITY

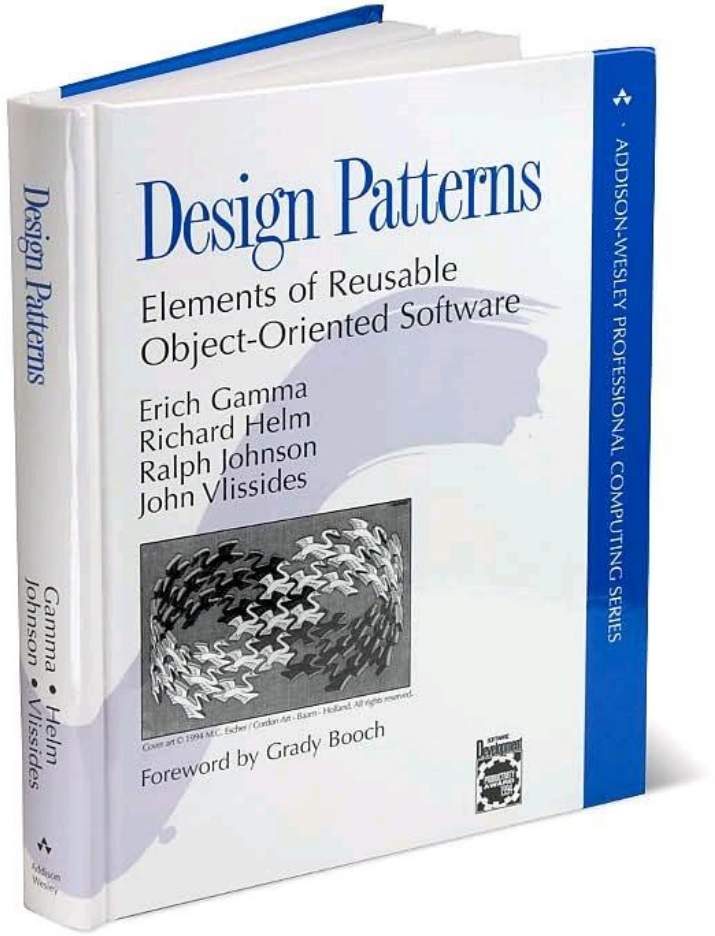
CSC3209 Software architecture and Design Patterns

Lecture13: Design Patterns Introduction

Subject Lecturer: Dr. Muhammed Basheer Jasser

References

- Materials from
  - Gamma, Erich. Design patterns: elements of reusable object-oriented software. Pearson Education India, 1995.



Lecture Outline

- What is a Design Pattern
- Design Pattern Catalogue
  - Creational Patterns
  - Structural Patterns
  - Behavioural Patterns

What is a Design Pattern?

- Design patterns make it easier to reuse successful designs and architectures.
- Expressing proven techniques as design patterns makes them more accessible to developers of new systems.
- Design patterns help you choose design alternatives that make a system reusable and avoid alternatives that compromise reusability.
- Design patterns can even improve the documentation and maintenance of existing systems by furnishing an explicit specification of class and object interactions and their underlying intent.
- Put simply, design patterns help a designer get a design "right" faster.

Main Elements of Design Patterns

- The **pattern name** is a handle we can use to describe a design problem, its solutions, and consequences in a word or two.
- The **problem** describes when to apply the pattern. It explains the problem and its context. It might describe specific design problems such as how to represent algorithms as objects. It might describe class or object structures that are symptomatic of an inflexible design. Sometimes the problem will include a list of conditions that must be met before it makes sense to apply the pattern.
- The **solution** describes the elements that make up the design, their relationships, responsibilities, and collaborations. The solution doesn't describe a particular concrete design or implementation, because a pattern is like a template that can be applied in many different situations.
- The **consequences** of applying the pattern
  - Time and space trade off
  - Language and implementation issues
  - Effects on flexibility, extensibility, portability

Describing Design Patterns (1)

- We describe design patterns using a consistent format. Each pattern is divided into sections according to the following template.
  1. **Pattern Name and Classification**
  2. **Intent**:
  3. **Also Known As**
  4. **Motivation**
  5. **Applicability**
  6. **Structure**
  7. **Participants**
  8. **Collaborations**
  9. **Consequences**
  10. **Implementation**
  11. **Sample Code**
  12. **Known Uses**
  13. **Related Patterns**

Describing Design Patterns (2)

1. **Pattern Name and Classification**
2. **Intent**: A short statement that answers the following questions: What does the design pattern do? What is its rationale and intent? What particular design issue or problem does it address?
3. **Also Known As**: Other well-known names for the pattern, if any.
4. **Motivation**: A scenario that illustrates a design problem and how the class and object structures in the pattern solve the problem. The scenario will help you understand the more abstract description of the pattern that follows.
5. **Applicability:** What are the situations in which the design pattern can be applied? What are examples of poor designs that the pattern can address? How can you recognize these situations?

Describing Design Patterns (3)

6. **Structure**: A graphical representation of the classes in the pattern using a notation based on the Object Modeling Technique (OMT). We also use interaction diagrams to illustrate sequences of requests and collaborations between objects.

7. **Participants**: The classes and/or objects participating in the design pattern and their responsibilities.

8. **Collaborations**: How the participants collaborate to carry out their responsibilities.

9. **Consequences**: How does the pattern support its objectives? What are the trade-offs and results of using the pattern? What aspect of system structure does it let you vary independently?

Describing Design Patterns (4)

10. **Implementation**: What pitfalls, hints, or techniques should you be aware of when implementing the pattern? Are there language-specific issues?

11. **Sample Code**: Code fragments that illustrate how you might implement the pattern in a programming language.

12. **Known Uses**: Examples of the pattern found in real systems.

13. **Related Patterns**: What design patterns are closely related to this one? What are the important differences? With which other patterns should this one be used?

Creational Patterns

1. Abstract factory
2. Builder
3. Factory method
4. Prototype
5. Singleton

Structural Patterns

1. Adapter
2. Bridge
3. Composite
4. Decorator
5. Façade
6. Flyweight
7. Proxy

Behavioral Patterns

1. Chain of Responsibility
2. Command
3. Interpreter
4. Iterator
5. Mediator
6. Memento
7. Observer
8. State
9. Strategy
10. Template method
11. Visitor